

```
somergroup1: foo
somergruop2: bar
```

This method can be extended to LDAP/SQL authentication as well (using the appropriate Apache modules, like *mod_authnz_ldap*, *mod_dbd* and *mod_authn_dbd*). I won't go into the details of these in this article as there is sufficient documentation available online.

So let's say we write a Lua script named *authenticate.lua*. This is how you configure Nginx to execute the Lua script for checking access rights to a path */restricted*:

```
location ~ ^/restricted {
    set $user_group 'somergruop1';
    access_by_lua_file '/etc/nginx/authenticate.lua';
}
```

The Lua script to make it happen is given below:

```
-- basic configuration of the script

local cookie_domain = ".yourdomain.com"
local db_username = "dbuser"
local db_password = "dbpassword"
local db_socket = "/tmp/mysql.sock"
local db_name = "dbname"

-- end configuration

local session = require "resty.session".open{ cookie = {
domain = cookie_domain } }
local remote_password

if ngx.var.http_authorization then
    local tmp = ngx.var.http_authorization
    tmp = tmp:sub(tmp:find(' ') + 1)
    tmp = ngx.decode_base64(tmp)
    remote_password = tmp:sub(tmp:find(':') + 1)
end

function authentication_prompt()
    session.data.valid_user = false
    session.data.user_group = nil
    session:save()
    ngx.header.www_authenticate = "Basic realm=Restricted"
    ngx.exit(401)
end

function authenticate(user, password, group)
    local mysql = require "resty.mysql"
    local db, err, errno, sqlstate, res, ok

    db = mysql:new()
    if not db then
```

```
        ngx.log(ngx.ERR, "Failed to create mysql object")
        ngx.exit(500)
    end

    db:set_timeout(2000)
    ok, err, errno, sqlstate = db:connect{
        path = db_socket,
        database = db_name,
        user = db_username,
        password = db_password
    }

    if not ok then
        ngx.log(ngx.ERR, "Unable to connect to database: ",
err, ":", ":", errno, ":", ":", sqlstate)
        ngx.exit(500)
    end

    user = ngx.quote_sql_str(user)
    password = ngx.quote_sql_str(password)
    local query = "select 1 from http_users where username =
%s and password = SHA2(%s, 224) and (find_in_set('superadmin',
groups) > 0 or find_in_set('%s', groups) > 0)"
    query = string.format(query, user, password, group);
    res, err, errno, sqlstate = db:query(query)

    if res and res[1] then
        session.data.valid_user = true
        session.data.user_group = group
        session:save()
    else
        authentication_prompt()
    end
end

if session.present and (session.data.valid_user and session.
data.user_group == ngx.var.user_group) then
    return
elseif ngx.var.remote_user and remote_password then
    authenticate(ngx.var.remote_user, remote_password, ngx.
var.user_group)
else
    authentication_prompt()
end
end
```

The group authentication script looks for users and groups in a table called *http_users*. Since this is a script, you can modify the way users are searched for in the database or change the database altogether! The Lua modules required to run this script are *resty.mysql*, *resty.session*, *resty.string* and *cjson*. Though the passwords are stored in the database as a SHA224 hash, the comparison of the password is done by the database itself. I did not convert the password to hash before sending it to the database, so you could review this in